

Hermes Commander — Branch Specification v1.0

1. Decision

Create **Hermes Commander** as a Hermes-first mobile product branched from ClawCommander.

This should proceed before more ClawCommander expansion because the current code already contains most Hermes features. The branch can become a focused daily-driver app without carrying OpenClaw, local models, Academy Mode, Life Architect, or Paperclip baggage.

The goal is not to make a generic agent platform. The goal is:

A native mobile command centre for Hermes Agent, with the strongest practical mobile version of the Hermes WebUI experience.

2. Product Scope

2.1 In Scope

Feature	Main Surface
Hermes chat via REST and ACP	Chat
ACP streaming, tool activity cards, approvals, cancel	Chat
Hermes sessions	Control
Hermes memory: MEMORY.md, USER.md, SOUL.md, § entry editor	Control
Hermes cron jobs	Control
Hermes skills	Control
Hermes logs	Control
Hermes analytics, cost, token charts, insights	Control
Approvals bell and pending approval panel	Control
Capability gates	Control
Swarm compose	Swarm
Swarm mission tree	Swarm
Office view	Swarm
Osiris World Intelligence	Intel
RECON toolkit: DNS, WHOIS, SSL, IP intel, CVE lookup	Intel

Feature	Main Surface
Focus Sound Player	Ambient
World Radio / Radio Garden globe	Ambient
Voice input	Throughout
Supertonic TTS	Throughout
Hermes WebUI visual parity	Chat and Control
AgentMemory VPS integration	Control
Open-Notebook VPS integration	Control

2.2 Out of Scope

Feature	Keep In
OpenClaw WebSocket, Mission Control, agents, channels, devices	ClawCommander
OpenClaw SSH diagnostics	ClawCommander
Local GGUF models	ClawCommander
flama / llamadart local inference	ClawCommander
On-device RAG and knowledge base	ClawCommander
Academy Mode	ClawCommander
Life Architect / GROW	ClawCommander
Paperclip Company OS	ClawCommander
ClawHub / OpenClaw skill marketplace	ClawCommander

3. Repository Strategy

Use a branch first, not a separate repo yet.

```
git checkout main
git pull
git checkout -b hermes-commander
```

Keep the GitHub repo where it is for now.

Recommended branch model:

```
main          -> ClawCommander full platform
hermes-commander -> Hermes Commander product shell
```

Do not hard fork yet. A separate repo only makes sense after Hermes Commander has its own release cycle, icon set, app store listing, and issue backlog.

4. Build Flavor Setup

4.1 New App Flavor File

Create:

```
lib/app/app_flavor.dart
```

```
library;

enum AppFlavor {
  clawCommander,
  hermesCommander,
}

const _rawFlavor = String.fromEnvironment(
  'APP_FLAVOR',
  defaultValue: 'clawCommander',
);

const AppFlavor kAppFlavor = _rawFlavor == 'hermesCommander'
  ? AppFlavor.hermesCommander
  : AppFlavor.clawCommander;

extension AppFlavorConfig on AppFlavor {
  String get appName => switch (this) {
    AppFlavor.clawCommander => 'ClawCommander',
    AppFlavor.hermesCommander => 'Hermes Commander',
  };

  String get shortName => switch (this) {
    AppFlavor.clawCommander => 'Claw',
    AppFlavor.hermesCommander => 'Hermes',
  };

  bool get isHermesOnly => this == AppFlavor.hermesCommander;
}
```

4.2 Run Commands

```
# ClawCommander
flutter run --dart-define=APP_FLAVOR=clawCommander

# Hermes Commander
flutter run --dart-define=APP_FLAVOR=hermesCommander
```

4.3 Android Product Flavors

Update:

```
android/app/build.gradle.kts
```

Add product flavors:

```
android {
    namespace = "com.carmen.clawcommander"

    flavorDimensions += "app"

    productFlavors {
        create("claw") {
            dimension = "app"
            applicationId = "com.carmen.clawcommander"
            resValue("string", "app_name", "ClawCommander")
        }

        create("hermes") {
            dimension = "app"
            applicationId = "com.carmen.hermescommander"
            resValue("string", "app_name", "Hermes Commander")
        }
    }
}
```

Update Android manifest:

```
android:label="@string/app_name"
```

Build:

```
flutter build apk --flavor hermes --dart-define=APP_FLAVOR=hermesCommander
```

4.4 iOS Bundle ID

When iOS is active, add separate bundle IDs:

Flavor	Bundle ID
ClawCommander	com.carmen.clawcommander
Hermes Commander	com.carmen.hermescommander

Update Info.plist to read display name from build settings, not a hardcoded string.

5. Bottom Navigation

Hermes Commander has exactly five tabs:

 Chat |  Control |  Swarm |  Intel |  Ambient

No Memory bottom tab. No Skills bottom tab. Memory and Skills live inside Control because they are Hermes management surfaces.

6. Router Specification

Update:

```
lib/app/router.dart
```

6.1 Hermes Commander Route Tree

```
/
└─ ChatScreen

/control
├─ HermesManagementScreen
├─ /control/sessions
├─ /control/sessions/:id
├─ /control/memory
├─ /control/cron
├─ /control/skills
└─ /control/logs
```

```

├─ /control/analytics
├─ /control/approvals
├─ /control/agent-memory
├─ /control/notebook

/swarm
├─ SwarmMonitorScreen
├─ /swarm/compose
├─ /office

/intel
├─ WorldIntelligenceScreen
├─ /intel/recon

/ambient
├─ AmbientScreen

/settings
├─ SettingsScreen

```

6.2 Remove From Hermes Commander Shell

Remove these imports from the Hermes Commander route build:

```

features/academy/*
features/life_architect/*
features/mission_control/*
features/memory/*
features/skills/clawhub_browser.dart
features/company/*
features/knowledge_base/*
features/onboarding/model_download.dart
features/settings/model_config.dart
features/settings/models_screen.dart
features/settings/devices_screen.dart
features/settings/gateway_config.dart
features/settings/gateway_logs_screen.dart
features/settings/device_identity_settings.dart

```

Do not delete the files from the branch immediately. First gate them out of the router and settings. Delete after the Hermes Commander build passes.

6.3 App Shell Logic

Create a flavor-specific branch list.

```

List<StatefulShellBranch> buildShellBranches() {
  if (kAppFlavor.isHermesOnly) {
    return [
      chatBranch(),
      hermesControlBranch(),
      swarmBranch(),
      intelBranch(),
      ambientBranch(),
    ];
  }

  return [
    chatBranch(),
    clawControlBranch(),
    memoryBranch(),
    skillsBranch(),
    ambientBranch(),
  ];
}

```

Also create flavor-specific nav destinations:

```

List<NavigationDestination> buildDestinations(int approvalCount) {
  if (kAppFlavor.isHermesOnly) {
    return [
      const NavigationDestination(
        icon: Icon(Icons.chat_outlined),
        selectedIcon: Icon(Icons.chat),
        label: 'Chat',
      ),
      NavigationDestination(
        icon: NavIconWithBadge(
          icon: Icons.tune_outlined,
          count: approvalCount,
        ),
        selectedIcon: NavIconWithBadge(
          icon: Icons.tune,
          count: approvalCount,
        ),
        label: 'Control',
      ),
      const NavigationDestination(
        icon: Icon(Icons.account_tree_outlined),
        selectedIcon: Icon(Icons.account_tree),
        label: 'Swarm',
      ),
    ],
  }
}

```

```

    const NavigationDestination(
      icon: Icon(Icons.public_outlined),
      selectedIcon: Icon(Icons.public),
      label: 'Intel',
    ),
    const NavigationDestination(
      icon: Icon(Icons.headphones_outlined),
      selectedIcon: Icon(Icons.headphones),
      label: 'Ambient',
    ),
  ];
}

return clawCommanderDestinations(approvalCount);
}

```

7. Server Provider Simplification

Update:

```
lib/data/providers/server_providers.dart
```

For Hermes Commander, the active server is always Hermes.

Keep the enum only if shared code still needs it:

```
enum ActiveServer { openclaw, hermes, local }
```

But change provider behavior:

```

final activeServerProvider = StateProvider<ActiveServer>((ref) {
  if (kAppFlavor.isHermesOnly) return ActiveServer.hermes;

  // Existing ClawCommander detection logic
  return detectActiveServer(ref);
});

```

Update setter:

```

Future<void> setActiveServer(WidgetRef ref, ActiveServer server) async {
  if (kAppFlavor.isHermesOnly && server != ActiveServer.hermes) {
    return;
  }
}

```



```

    }

    await ref.read(sharedPrefsProvider).setString('active_server', server.name);
    ref.read(activeServerProvider.notifier).state = server;
  }

```

8. Chat Mode Simplification

Update:

```
lib/data/providers/chat_mode_providers.dart
```

Hermes Commander should not show Local/OpenClaw mode options.

```

final chatModeProvider = StateProvider<ChatMode>((ref) {
  if (kAppFlavor.isHermesOnly) return ChatMode.hermes;

  // Existing ClawCommander behavior
});

```

Update `ChatModeSelector` :

- In Hermes Commander, hide the mode selector.
- Replace it with a small transport chip:
 - Hermes · ACP
 - Hermes · REST
 - SSH disconnected
 - Approvals pending

The transport chip should not imply multiple agents. It only shows the Hermes connection path.

9. Capability Gates

Hermes Commander still needs capability gates, but they become simpler.

```

class ServerCapabilities {
  final bool hasHermesRest;
  final bool hasHermesAcp;
  final bool hasSsh;
  final bool hasSessions;
  final bool hasMemory;
  final bool hasCron;
  final bool hasSkills;
}

```

```

final bool hasLogs;
final bool hasAnalytics;
final bool hasApprovals;
final bool hasSwarm;
final bool hasOsiris;
final bool hasAmbient;

const ServerCapabilities({
  this.hasHermesRest = false,
  this.hasHermesAcp = false,
  this.hasSsh = false,
  this.hasSessions = false,
  this.hasMemory = false,
  this.hasCron = false,
  this.hasSkills = false,
  this.hasLogs = false,
  this.hasAnalytics = false,
  this.hasApprovals = false,
  this.hasSwarm = false,
  this.hasOsiris = false,
  this.hasAmbient = true,
});
}

```

Rules:

Hermes REST configured -> Chat basic
 SSH configured -> Control sessions, memory, cron, skills, logs, analytics
 ACP available -> full tool streaming, approvals, cancel, richer chat
 Osiris URL configured -> Intel live data and RECON
 Ambient -> always available

10. Branding

10.1 App Name

Item	Value
Display name	Hermes Commander
Internal flavor	hermesCommander
Android application ID	com.carmen.hermescommander
iOS bundle ID	com.carmen.hermescommander
Dart flavor define	APP_FLAVOR=hermesCommander

10.2 Theme Direction

Use a Hermes/WebUI-inspired dark command-console style.

Keep:

- JetBrains Mono
- dark surfaces
- compact cards
- terminal-style tool activity
- soft borders
- high-contrast status chips

Adjust:

Current ClawCommander	Hermes Commander
Lobster red identity	Hermes purple / indigo
Bronze highlights	violet, cyan, slate
OpenClaw badges	Hermes status chips
broad platform feel	focused operator console

Suggested palette:

```
class HermesCommanderPalette {  
    static const background = Color(0xFF080A12);  
    static const surface = Color(0xFF10131F);  
    static const surfaceHigh = Color(0xFF171B2B);  
    static const outline = Color(0xFF2A3045);  
  
    static const hermesPurple = Color(0xFF8B5CF6);  
    static const commandCyan = Color(0xFF22D3EE);  
    static const success = Color(0xFF22C55E);  
    static const warning = Color(0xFFFF59E0B);  
    static const danger = Color(0xFFEF4444);  
  
    static const text = Color(0xFFE5E7EB);  
    static const muted = Color(0xFF94A3B8);  
}
```

10.3 Splash and Icon

Create:

```
assets/icon/hermescommander_icon.png
assets/icon/hermescommander_icon_foreground.png
```

Icon concept:

- Hermes wing / helmet / command glyph
- dark background
- purple/cyan foreground
- no claw imagery

11. Hermes WebUI Mobile Parity

The target is not to copy WebUI one-for-one. The target is to translate its operating model to mobile.

11.1 WebUI Concept → Mobile Translation

Hermes WebUI	Hermes Commander Mobile
Left session sidebar	Chat drawer / session bottom sheet
Center chat	Chat tab
Right workspace file panel	Slide-up workspace/file preview sheet
Composer footer model/profile controls	Composer footer chips
Circular context ring	Compact context ring beside composer or header
Control Center modal	Control tab + quick settings sheet
Tool cards	TUI Activity Card
Projects/tags	Session metadata chips
Workspace file browser	Control → Workspace / Notebook
Mermaid rendering	Chat bubble renderer
KaTeX rendering	Chat bubble renderer
Syntax highlighting	Chat bubble renderer
Edit + regenerate	Message actions bar
Export/import sessions	Session detail menu
Slash commands	Composer command palette

11.2 Chat Screen Layout

Chat header:

```
[Hermes Commander]      [context ring] [approvals bell] [settings]
```

Message stream:

```
User bubble
Assistant bubble
  └─ Thinking panel
  └─ TUI Activity Card
  └─ Markdown content
  └─ Message actions
```

Composer:

```
[+] [text input.....] [mic] [send]
[Hermes ACP] [profile] [workspace] [model] [tokens]
```

11.3 Context Ring

Add widget:

```
lib/shared/widgets/context_ring.dart
```

Inputs:

```
class ContextRing extends StatelessWidget {
  final int usedTokens;
  final int maxTokens;
  final double? estimatedCost;
  final String? model;
}
```

Behavior:

- green below 60%
- amber 60–85%
- red above 85%
- tap opens session context sheet
- sheet shows input/output/cache tokens, cost, model, provider, session ID

11.4 Mermaid Support

Add a fenced-code detector:

```
```mermaid
```

```
graph TD
A --> B
```

Render as:

- initial version: monospaced card with “Open diagram” action
- later version: WebView or native graph renderer

Do not block the branch on perfect Mermaid rendering. Ship readable fallback first.

### 11.5 Edit + Regenerate

Extend `MessageActionBar`:

User messages:

- Copy
- Edit
- Resend

Assistant messages:

- Copy
- Regenerate
- Continue
- Read aloud

Regenerate should:

1. truncate messages after selected user turn
2. send edited text through Hermes
3. preserve old branch if “fork session” is selected

## 12. Control Tab

Control is the Hermes management hub.

Top-level cards:

```
```text
Connection
Sessions
Memory
Cron
Skills
Logs
Analytics
Approvals
AgentMemory
Open-Notebook
```

12.1 Control Home

Cards:

- REST status
- SSH status
- ACP status
- current profile
- active model
- pending approvals
- today's tokens
- today's estimated cost
- cron failures
- last session

12.2 Sessions

Use existing Hermes sessions screen.

Required polish:

- grouped by Today / Yesterday / This Week / Earlier
- title, source, model, cost
- badges for CLI, REST, ACP, cron, swarm
- search
- export transcript
- import from JSON later

12.3 Memory

Use Hermes memory editor.

Required:

- never expose raw `$` delimiters as normal user text

- cards per memory entry
- add/edit/delete entry
- save with delimiter restoration
- tabs for MEMORY.md, USER.md, SOUL.md

12.4 Cron

Use Hermes cron screen.

Required:

- list jobs
- enable/disable
- run now
- create/delete
- schedule builder
- delivery target selector if available

12.5 Skills

Hermes skills only.

Do not show ClawHub.

Required:

- installed skills
- generated/self-improving skills
- skill detail
- enable/disable if supported
- file path/source
- last modified

12.6 Logs

Tabs:

- agent log
- errors log
- gateway log
- ACP session log

Use tail-over-SSH where possible.

12.7 Analytics

Cards:

- 7-day token chart
- per-model cost ledger
- sessions by source
- cron success/failure
- insights summary

13. Swarm Tab

Keep the existing Swarm and Office View work.

Routes:

```
/swarm  
/swarm/compose  
/office
```

Swarm tab contents:

- active missions
- mission tree
- delegated subagents
- status timeline
- Office View shortcut

The Swarm tab must be clearly framed as Hermes delegation / conductor work, not a generic Paperclip company view.

14. Intel Tab

Create:

```
lib/features/intel/intel_screen.dart  
lib/features/intel/world_intelligence_screen.dart  
lib/features/intel/recon_panel.dart  
lib/core/intelligence/osiris_client.dart  
lib/core/intelligence/intelligence_models.dart  
lib/data/providers/intelligence_providers.dart
```

14.1 Intel Home

Sections:

- World Intelligence map
- RECON toolkit
- Recent lookups
- Hermes MCP status
- Osiris service health

14.2 World Intelligence Layers

Layers:

- earthquakes
- flights
- fires
- news
- conflict zones
- satellites if available

14.3 RECON Toolkit

Tools:

- DNS lookup
- WHOIS lookup
- SSL inspection
- IP intelligence
- CVE lookup

Safety boundary:

- Label RECON as defensive / owned-target analysis.
- Do not encourage scanning systems the user does not own or have permission to test.
- Avoid offensive exploitation workflows.

15. Ambient Tab

Keep the current Ambient tab.

Routes:

```
/ambient
```

Sections:

- Focus Sounds
- World Radio

Keep persistent mini-player above the bottom nav.

In Hermes Commander, Ambient is not “Hermes management.” It is a daily-driver support layer.

16. Settings

Hermes Commander settings should only show:

```
Hermes REST
Hermes SSH
Hermes ACP
Hermes profiles
Osiris
Voice input
Supertonic TTS
Ambient audio
Theme
Security / biometric lock
Backup / restore
About
```

Remove:

```
OpenClaw gateway
Device identity pairing
OpenClaw devices
OpenClaw logs
Local models
Model downloads
HuggingFace token
RAG storage
Academy Mode
Life Architect
Paperclip company
LAN discovery
ClawHub
```

17. File Change List

17.1 Add

```
lib/app/app_flavor.dart
lib/app/hermes_commander_shell.dart
lib/features/intel/intel_screen.dart
lib/features/intel/world_intelligence_screen.dart
lib/features/intel/recon_panel.dart
lib/core/intelligence/osiris_client.dart
lib/core/intelligence/intelligence_models.dart
lib/data/providers/intelligence_providers.dart
lib/shared/widgets/context_ring.dart
assets/icon/hermescommander_icon.png
assets/icon/hermescommander_icon_foreground.png
```

17.2 Modify

```
pubspec.yaml
android/app/build.gradle.kts
android/app/src/main/AndroidManifest.xml
ios/Runner/Info.plist
lib/main.dart
lib/app/app.dart
lib/app/router.dart
lib/app/theme.dart
lib/data/providers/server_providers.dart
lib/data/providers/chat_mode_providers.dart
lib/data/providers/capability_providers.dart
lib/features/chat/chat_screen.dart
lib/features/chat/chat_bubble.dart
lib/features/chat/chat_mode_selector.dart
lib/features/settings/settings_screen.dart
lib/features/hermes/hermes_management_screen.dart
```

17.3 Hide From Router First

Do not delete immediately.

```
lib/features/academy/
lib/features/life_architect/
lib/features/mission_control/
lib/features/company/
lib/features/knowledge_base/
```

```
lib/core/llm/  
lib/core/local_agent/  
lib/core/rag/  
lib/core/openclaw/  
lib/core/gateway/
```

17.4 Delete Later After Build Passes

Delete only after no imports remain:

```
OpenClaw-only screens  
Paperclip-only screens  
Academy screens  
Life Architect screens  
Local model screens  
RAG screens  
OpenClaw-only providers  
OpenClaw-only models
```

18. Implementation Order

Phase 0 — Safety and Baseline

1. Create branch.
2. Build current app.
3. Run `flutter analyze`.
4. Confirm no hardcoded production secrets.
5. Commit baseline.

Phase 1 — Flavor and Branding

1. Add `app_flavor.dart`.
2. Add Android product flavors.
3. Change manifest label to `@string/app_name`.
4. Add Hermes Commander icon placeholders.
5. Build Hermes flavor.

Phase 2 — Hermes-Only Server Scope

1. Force `activeServerProvider` to Hermes in Hermes flavor.
2. Force `chatModeProvider` to Hermes.
3. Hide chat mode selector.
4. Remove server switcher.
5. Update settings to Hermes-only.

Phase 3 — Router Split

1. Build Hermes Commander 5-tab shell.
2. Add Chat, Control, Swarm, Intel, Ambient branches.
3. Move Memory and Skills into Control.
4. Remove ClawCommander-only tabs.
5. Confirm back navigation.

Phase 4 — Control Consolidation

1. Make Hermes Management the Control home.
2. Add sessions, memory, cron, skills, logs, analytics.
3. Add approvals panel.
4. Add AgentMemory placeholder.
5. Add Open-Notebook placeholder.

Phase 5 — Intel

1. Add Osiris client.
2. Add Osiris settings.
3. Add world intelligence screen.
4. Add RECON panel.
5. Add Hermes MCP status card.

Phase 6 — WebUI Parity Sprint

1. Context ring.
2. Composer footer chips.
3. Chat drawer / session sheet.
4. Workspace file preview sheet.
5. Mermaid fallback.
6. Edit + regenerate.
7. Export transcript.

Phase 7 — Polish and QA

1. Physical Android test.
2. Tailscale test.
3. REST-only mode test.
4. SSH-enabled mode test.
5. ACP mode test.
6. Osiris unavailable state.
7. Ambient background audio test.
8. Release APK build.

19. Acceptance Criteria

Hermes Commander is ready when:

- app launches as “Hermes Commander”
- package ID is separate from ClawCommander
- bottom nav shows only Chat, Control, Swarm, Intel, Ambient
- no OpenClaw UI appears anywhere
- no local model UI appears anywhere
- Chat defaults to Hermes
- Control shows Hermes management only
- Memory and Skills are Hermes-scoped
- Swarm works without Paperclip
- Intel works with Osiris configured and degrades cleanly without it
- Ambient works without VPS access
- settings are Hermes-only
- no hardcoded secrets exist
- release APK builds successfully

20. Non-Goals

Do not build these in Hermes Commander:

- OpenClaw compatibility
- local inference
- model downloads
- on-device RAG
- Academy
- Life Architect
- Paperclip Company OS
- ClawHub marketplace
- public multi-user collaboration
- offensive security tooling

21. Summary

Hermes Commander should feel like Hermes WebUI became a native mobile command app.

ClawCommander remains the broader multi-agent platform.

Hermes Commander becomes the focused daily-driver app for people who live inside Hermes Agent.